
SPRITES ARE CHEAP

Writing a retro-style game in JavaScript

Is writing a 2D game easier
nowadays than 30-40 years ago?

Do current tools allow us to
easily create and manage
sprites?

Are sprites cheap nowadays?

SCHEDULE

1. Introduction - what are sprites & JS
2. PHASER library
3. Phaser setup
4. Live coding:
 - static sprite
 - multiple sprites
 - animated sprite
 - sprite controls
 - collisions
 - score
5. Summary

1. Java Script

- programming language used to create interactive websites
- runs in web browsers to add dynamic content and features
- can also be used for server-side development with tools like Node.js
- created by Brendan Eich in 1995 (as LiveScript) and has become a fundamental technology of the web
- easily available from browser console

2.PHASER

- open-source JavaScript library used to create 2D games for web browsers
- framework for rendering graphics, handling input, and managing game physics
- known for its ease of use, extensive documentation, and active community support.

PHASER SETUP

2.1 Create an HTML page

2.2 Include this link to phaser.js
in your page

<https://cdn.jsdelivr.net/npm/phaserc3/dist/phaser.js>

```
<html>
  <head>
    <script src="https://cdn.jsdelivr.net/npm/phaserc3/dist/phaser.js"></script>
  </head>
</html>
```

2.3 Create an HTML div to display
the game

```
<body>
  <div id="game-container"></div>
</body>
```

2.4 Add JS section with phaser parameters

```
<script>
  const config = {
    type: Phaser.CANVAS, // rendering style (WebGL or Canvas)
    width: 800,
    height: 600,
    parent: 'game-container'
  };
</script>
```

2.5 Create game object

```
const game = new Phaser.Game(config);
```

Full code so far:

```
1  <html>
2  <head>
3    <script src="https://cdn.jsdelivr.net/npm/phaser@3/dist/phaser.js"></script>
4  </head>
5  <body>
6    <div id="game-container"></div>
7    <script>
8      const config = {
9        type: Phaser.AUTO, // rendering style (WebGL or Canvas)
10       width: 800,
11       height: 600,
12       parent: 'game-container'
13     };
14
15     const game = new Phaser.Game(config);
16   </script>
17 </body>
18 </html>
```


2.6 Add default scene and functions preload (), create() and update

```
function preload(){  
}  
  
function create(){  
  this.add.text(20, 20, 'Super fajna gra!', { font: '24px Arial', fill: 'ffffff' });  
}  
  
function update(){  
}
```

```
scene: {  
  preload: preload,  
  create: create,  
  update: update  
}
```

2.7 Adding a sprite



Now look at the design of the balloon. The first series of 8 squares on the first row are all blank, therefore the bits are all 0FF giving a value of zero. The middle series on row 1 looks like this:

0000000 0111111 0000000

The third series of 8 squares on the first row also contains only blanks so it also equals zero. So the data for the first line is:

DATA 0, 127, 0

The three series of data that make up row two are calculated like this:

$$\begin{array}{l}
 \text{Series 1: } \begin{array}{|c|c|c|c|c|c|c|c|} \hline 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \hline \end{array} \quad \begin{array}{c} \uparrow \\ 128 \end{array} = 0 \\
 \text{Series 2: } \begin{array}{|c|c|c|c|c|c|c|c|} \hline 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ \hline \end{array} \quad \begin{array}{c} \uparrow \\ 64 \end{array} = 512 \\
 \text{Series 3: } \begin{array}{|c|c|c|c|c|c|c|c|} \hline 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \hline \end{array} \quad \begin{array}{c} \uparrow \\ 128 \end{array} = 0 \\
 \hline
 128 + 512 = 640
 \end{array}$$

The data for the second row is:

DATA 1, 255, 132

Use this method to convert the three series of 8 squares in each of the remaining rows. Once you have completed these calculations, you are ready to write a BASIC program to use the balloon (or any other shape) since the sprite has now been converted into values your computer can understand.

To demonstrate the use of sprites, type in the following program:

```

1  REM UP, UP, AND AWAY
2  PRINT "CLRHOME"
3  V = 53248 : REM START OF DISPLAY CHIP
4  POKE V + 254 : REM ENABLE SPRITE 2
5  POKE 2042,13 : REM SPRITE 2 DATA FROM BLOCK 13
6  FOR N = 0 TO 32 : READ D : POKE 832 + N,D : NEXT
7  FOR X = 0 TO 200
8  POKE V + 4,X : REM UPDATE X COORDINATES
9  POKE V + 5,X : REM UPDATE Y COORDINATES
10 NEXT X
11 GOTO 30
12 DATA 0,127,0,1,255,192,3,255,224,3,231,224
13 DATA 7,217,240,7,223,240,7,217,240,3,231,224
14 DATA 3,255,224,3,255,224,2,255,192,1,127,64
15 DATA 1,62,64,0,196,128,0,156,128,0,73,0,73,0
16 DATA 0,62,0,62,0,62,0,5,28,0

```

GETS 8th INFO FROM DATA

8th INFO FROM S

If you have entered everything correctly, when you type "RUN" and press RETURN, your balloon should sail across the screen. At this stage, you will not understand the meaning of much of the program but, as we explain each stage of sprite handling, we will use the program to illustrate each feature.

Line numbers 200-240 relate to the definition of your balloon and contain 21 sets of three values, i.e. one set for each row on your design chart.

Our sprite:

- No need to define individual pixels
- Use PNG, BMP, JPG pictures instead:

balloon.png



Simple code to add a sprite:

```
let sprite = null;

function preload() {
  this.load.image('balloon', 'balloon.png');
}

function create() {
  this.add.text(20, 20, 'Super fajna gra!', { font: '24px Arial', fill: 'ffffff' });
  sprite = this.add.sprite(200, 200, 'balloon');
}
```

Super fajna gra!



Create 12 sprites:

```
const spritesCount = 12;  
let sprites = [];
```

```
function create() {  
  this.add.text(20, 20, 'Super fajna gra!', { font: '24px Arial', fill: '#ffffff' });  
  
  for (let i = 0; i < spritesCount; i++){  
    let s = this.add.sprite(i, 50*i, 'balloon');  
    sprites.push(s);  
  }  
}
```

Let's get movin'...

```
function update() {  
  for (let i = 0; i < spritesCount; i++){  
    if (sprites[i].x < 800)  
      sprites[i].x += i; //(i+1) % 20;  
    else  
      sprites[i].x = 0;  
  }  
}
```

2.8 Introducing player

Animated player sprite




```
let player;

function preload() {
  this.load.image('balloon', 'balloon.png');

  this.load.spritesheet('digdug', 'empty-project/dig-dug.png', {
    frameWidth: 75,
    frameHeight: 75
  });
}
```

Animating player

```
function create() {  
  //.....  
  
  player = this.add.sprite(400, 300, 'digdug');  
  this.anims.create({  
    key: 'walk',  
    frames: this.anims.generateFrameNumbers('digdug', { start: 0, end: 3 }),  
    frameRate: 5,  
    repeat: -1  
  });  
  
  player.play('walk');  
}
```

Moving player

```
let cursors;  
  
function create(){  
    //....  
    cursors = this.input.keyboard.createCursorKeys();  
}
```

```
function update(){  
    //....  
    checkKeys();  
}
```

```
function checkKeys(){  
  if (cursors.left.isDown) {  
    player.x -= 1;  
  } else if (cursors.right.isDown) {  
    player.x += 1;  
  }  
  
  if (cursors.up.isDown) {  
    player.y -= 1;  
  } else if (cursors.down.isDown) {  
    player.y += 1;  
  }  
}
```

2.9 Collisions

```
function checkHit(sprite){  
    let d = Phaser.Math.Distance.Between(sprite.x + sprite.width/2, sprite.y + sprite.height/2  
        , player.x + player.width/2, player.y + player.height/2);  
  
    if (d < 20){  
        sprite.y = Math.pow(10,16);  
    }  
}
```

2.10 Score

```
<div id="score">SCORE: 0</div>
```

.....

```
function increaseScore(){  
  score++;  
  const scoreDiv = document.getElementById('score');  
  scoreDiv.innerText = 'SCORE: ' + score;  
}
```